LAB ASSIGNMENT -16.1

Humera Mahaveen
2303A53030
Batch-46

Task 1 – Student Information System

Prompt

Generate schema for Students, Courses, Enrollments with PK, FK and queries for insert, fetch courses, count students.

```
CREATE TABLE Students(student_id INT PRIMARY KEY, name VARCHAR(100), age INT);
CREATE TABLE Courses(course_id INT PRIMARY KEY, course_name VARCHAR(100));
CREATE TABLE Enrollments(enrollment_id INT PRIMARY KEY, student_id INT, course_id INT,
FOREIGN KEY(student_id) REFERENCES Students(student_id),
FOREIGN KEY(course_id) REFERENCES Courses(course_id));

INSERT INTO Students VALUES (1,'Akhil',20);

SELECT c.course_name FROM Courses c
JOIN Enrollments e ON c.course_id=e.course_id
WHERE e.student_id=1;

SELECT course_id, COUNT(student_id) FROM Enrollments GROUP BY course_id;
```

Task 2 – Hospital Management

Prompt

Create schema for Doctors, Patients, Appointments and queries for doctor appointments, patient history, patient count.

```
CREATE TABLE Doctors(doctor_id INT PRIMARY KEY, name VARCHAR(100), specialization
VARCHAR(100));
CREATE TABLE Patients(patient_id INT PRIMARY KEY, name VARCHAR(100), age INT);
CREATE TABLE Appointments(appointment_id INT PRIMARY KEY, doctor_id INT, patient_id
INT, appointment_date DATE,
FOREIGN KEY(doctor_id) REFERENCES Doctors(doctor_id),
FOREIGN KEY(patient_id) REFERENCES Patients(patient_id));
```

```
SELECT * FROM Appointments WHERE doctor_id=1;

SELECT * FROM Appointments WHERE patient_id=1;

SELECT doctor_id, COUNT(patient_id) FROM Appointments GROUP BY doctor_id;
```

Task 3 – Library Database

Prompt

Design schema for Books, Members, Loans and queries for issued books, overdue books, count per member.

```
CREATE TABLE Books(book_id INT PRIMARY KEY, title VARCHAR(100));
CREATE TABLE Members(member_id INT PRIMARY KEY, name VARCHAR(100));
CREATE TABLE Loans(loan_id INT PRIMARY KEY, book_id INT, member_id INT, loan_date
DATE, return_date DATE,
FOREIGN KEY(book_id) REFERENCES Books(book_id),
FOREIGN KEY(member_id) REFERENCES Members(member_id));

CREATE INDEX idx_loan_date ON Loans(loan_date);

SELECT b.title FROM Books b JOIN Loans l ON b.book_id=l.book_id WHERE l.return_date IS
NULL;

SELECT * FROM Loans WHERE return_date IS NULL AND loan_date < DATE_SUB(CURDATE(),
INTERVAL 30 DAY);

SELECT member_id, COUNT(book_id) FROM Loans GROUP BY member_id;
```

Task 4 – Online Shopping Database

Prompt

Create schema for Users, Products, Orders, OrderDetails and queries for user orders, most purchased product, revenue.

```
CREATE TABLE Users(user_id INT PRIMARY KEY, name VARCHAR(100));
CREATE TABLE Products(product_id INT PRIMARY KEY, product_name VARCHAR(100), price
DECIMAL(10,2));
```

```
CREATE TABLE Orders(order_id INT PRIMARY KEY, user_id INT, order_date DATE,
FOREIGN KEY(user_id) REFERENCES Users(user_id));
CREATE TABLE OrderDetails(order_detail_id INT PRIMARY KEY, order_id INT, product_id INT,
quantity INT,
FOREIGN KEY(order_id) REFERENCES Orders(order_id),
FOREIGN KEY(product_id) REFERENCES Products(product_id));

SELECT * FROM Orders WHERE user_id=1;

SELECT product_id, SUM(quantity) AS total FROM OrderDetails GROUP BY product_id
ORDER BY total DESC LIMIT 1;

SELECT SUM(p.price*od.quantity) FROM OrderDetails od
JOIN Products p ON od.product_id=p.product_id
JOIN Orders o ON od.order_id=o.order_id
WHERE MONTH(o.order_date)=3;
```

Task 5 – University Examination System

Prompt

Design schema for Students, Subjects, Exams, Results and queries for insert result, subject-wise marks, average marks.

```
CREATE TABLE Students(student_id INT PRIMARY KEY, name VARCHAR(100));
CREATE TABLE Subjects(subject_id INT PRIMARY KEY, subject_name VARCHAR(100));
CREATE TABLE Exams(exam_id INT PRIMARY KEY, subject_id INT, exam_date DATE,
FOREIGN KEY(subject_id) REFERENCES Subjects(subject_id));
CREATE TABLE Results(result_id INT PRIMARY KEY, student_id INT, exam_id INT, marks INT,
FOREIGN KEY(student_id) REFERENCES Students(student_id),
FOREIGN KEY(exam_id) REFERENCES Exams(exam_id));

INSERT INTO Results VALUES (1,1,1,85);

SELECT s.subject_name, r.marks FROM Results r
JOIN Exams e ON r.exam_id=e.exam_id
JOIN Subjects s ON e.subject_id=s.subject_id
WHERE r.student_id=1;

SELECT subject_id, AVG(marks) FROM Results r
JOIN Exams e ON r.exam_id=e.exam_id
GROUP BY subject_id;
```